МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №6

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод простых итераций

Студентка гр. 4342 Волкова В.Н.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Используя программы-функции ITER и ROUND из файла methods.cpp, найти корень уравнения $f(x) = 0$ с заданной точностью Eps методом простых итераций, исследовать скорость сходимости и обусловленность метода.

Основные теоретические положения.

Метод простых итераций основан на преобразовании исходного уравнения $f(x) = 0$ к эквивалентному виду $x = \varphi(x)$. После этого строится последовательность приближений $x_{+1} = \varphi(x)$, которая при выполнении условий сходимости стремится к точному корню уравнения.

Достаточным условием сходимости метода на отрезке [a, b] является существование такого числа q, что $|\varphi'(x)| \leq q < 1$ для всех $x \in [a, b]$. При $\varphi'(x) > 0$ последовательность приближений сходится к корню монотонно, а при $\varphi'(x) < 0$ — колеблется около него.

Скорость сходимости метода определяется величиной q: чем меньше q, тем быстрее убывает погрешность. Следовательно, для успешного применения метода решающим является удачный выбор функции $\varphi(x)$ в эквивалентном уравнении $x = \varphi(x)$.

В библиотеке methods.cpp для реализации метода используется программа-функция ITER. Она вычисляет последовательные приближения и завершает процесс, когда разность между соседними значениями становится достаточно малой. Для исследования влияния ошибок исходных данных применяется функция Round, которая моделирует округление значений с заданной точностью Delta.

Выполнение работы.

Вариант 2: функция $f(x) = \arccos(x^2) - x$

1. Отделим корень аналитически. Функция определена при $|x| \leq 1$. На промежутке [-1, 0] имеем $\arccos(x^2) \geq 0$ и $-x \geq 0$, следовательно, $f(x) = \arccos(x^2) - x > 0$. Значит, отрицательных корней уравнение не имеет.

Рассмотрим отрезок [0.7, 0.9]. На нем функция непрерывна, а ее производная имеет вид:

$$f'(x) = -2x / \sqrt{(1 - x^4)} - 1.$$

Для всех $x \in [0.7, 0.9]$ выполняется $f'(x) < 0$, поэтому функция строго убывает. Кроме того,

$$f(0.7) = \arccos(0.49) - 0.7 \approx 0.3587 > 0, \quad f(0.9) = \arccos(0.81) - 0.9 \approx -0.2734 < 0.$$

Следовательно, на отрезке [0.7, 0.9] существует единственный корень уравнения $f(x) = 0$.

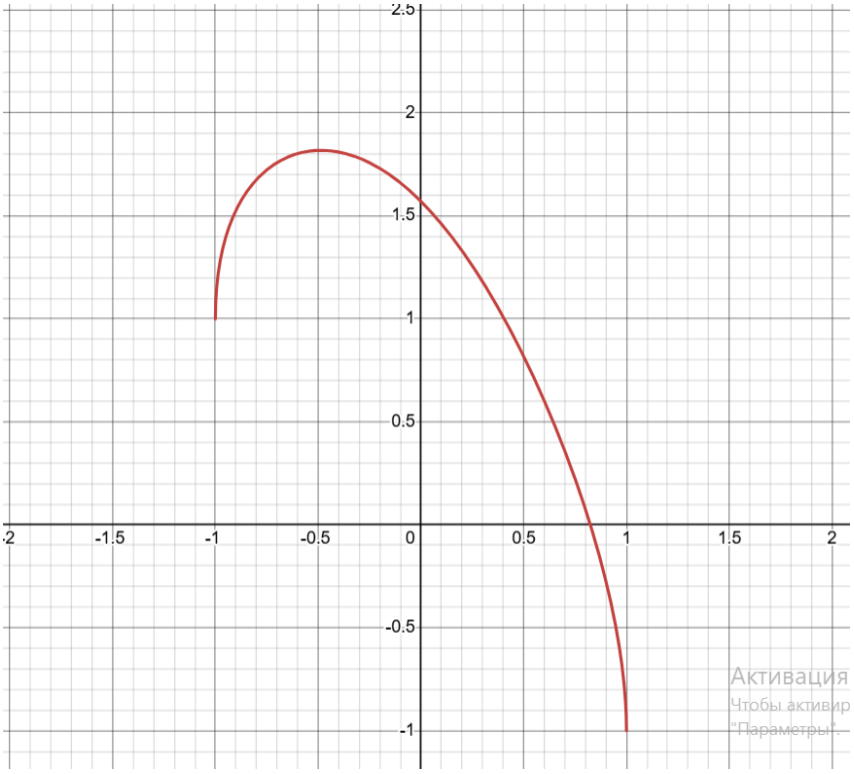

Рисунок 1 — график функции f(x)

2. Преобразуем исходное уравнение к виду $x = \varphi(x)$.

Хотя исходное уравнение

$$f(x) = \arccos(x^2) - x$$

формально уже можно записать в виде

$$x = \varphi(x), \qquad \varphi(x) = \arccos(x^2),$$

такой выбор функции итерации неудобен для метода простых итераций. Для сходимости метода необходимо, чтобы на выбранном отрезке выполнялось условие

$$|\varphi'(x)| \le q < 1.$$

В данном случае

$$\varphi(x) = -2x / \sqrt{(1 - x^4)} - 1, \quad |\varphi'(x)| = 2x / \sqrt{(1 - x^4)} - 1.$$

На отрезке [0,1] это условие не выполняется: при $x \to 1$ знаменатель стремится к нулю, поэтому $|\varphi'(x)|$ резко возрастает, а в самой точке корня $x^* \approx 0.82413$ имеем

$$|\varphi'(x^*)| \approx 2.25 > 1.$$

Следовательно, отображение $\varphi(x)=\arccos(x^2)$ не является сжимающим, и итерационный процесс не удовлетворяет достаточному условию сходимости. Поэтому для решения выбирается другое преобразование.

Из равенства $\arccos(x^2) = x$ получаем $x^2 = \cos x$. Так как искомый корень положителен, выбираем преобразование

$$x = \varphi(x) = \sqrt{(\cos x)}.$$

Для функции $\varphi(x)$ имеем производную

$$\varphi'(x) = -\sin x / (2\sqrt{(\cos x)}).$$

На отрезке [0.7, 0.9] выполнено условие $\varphi([0.7, 0.9]) \subset [0.7, 0.9]$, поскольку

$$\sqrt{(\cos 0.9)} \approx 0.7880, \quad \sqrt{(\cos 0.7)} \approx 0.8749.$$

Следовательно, все итерации, начиная с $x_0 \in [0.7, 0.9]$, остаются внутри выбранного интервала.

3. Проверим условие сходимости метода. На [0.7, 0.9] функция $|\varphi'(x)|$ возрастает, поэтому

$$q = \max |\varphi'(x)| = |\varphi'(0.9)| = \sin 0.9 / (2\sqrt{(\cos 0.9)}) \approx 0.4968 < 1.$$

Тем самым достаточное условие сходимости выполнено. Поскольку $\varphi'(x) < 0$ на [0.7, 0.9], последовательность приближений должна сходиться к корню с колебаниями относительно точного значения.

В качестве начального приближения выберем $x_0 = 0.8$, так как эта точка принадлежит отрезку [0.7, 0.9] и находится достаточно близко к искомому корню.

4. В файле lb6.cpp переопределена функция F(x), которая возвращает значение φ(x) = √(cos x), так как именно функция φ(x) должна передаваться в ITER при решении уравнения x = φ(x). Дополнительно реализована функция F1(x), возвращающая φ′(x); она нужна для теоретической проверки условия сходимости.

Для анализа результата в программе также введена вспомогательная функция Equation(x), вычисляющая значение исходной функции arccos(x²) - x. Если задана точность global_delta, то значение φ(x) перед возвратом округляется с помощью функции Round.

В функции main() вычисления разделены на две части: исследование скорости сходимости по зависимости числа итераций N от точности Eps и исследование обусловленности по влиянию Delta на найденный корень.

5. Проведены вычисления по программе. Для случая без округления получены результаты, на основе которых построен график зависимости N от Eps.

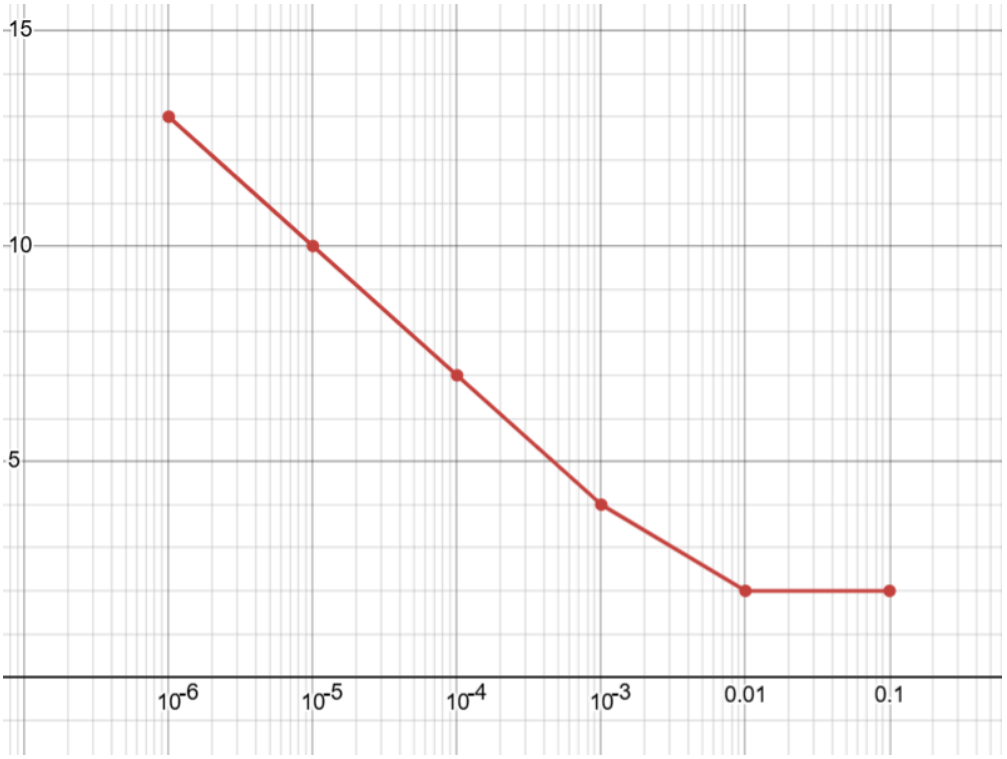

Рисунок 2 — график зависимости N от Eps

Из графика видно, что метод простых итераций сходится заметно медленнее метода Ньютона: при уменьшении Eps от 10^{-1} до 10^{-6} число итераций

увеличилось с 2 до 13. Такое поведение соответствует линейной сходимости метода.

Последовательность приближений при $x_0 = 0.8$ имеет вид $x_1 \approx 0.8346895886$, $x_2 \approx 0.8193947505$, $x_3 \approx 0.8262345962$, $x_4 \approx 0.8231947393$, … Приближения поочередно располагаются по разные стороны от корня, что полностью согласуется с условием $\varphi'(x) < 0$.

Если принять точное значение корня $x^* \approx 0.8241323123$, то отношения соседних ошибок быстро стабилизируются около $|\varphi'(x^*)| \approx 0.445$. Следовательно, на каждом шаге ошибка уменьшается примерно в 2.2 раза, что также подтверждает линейный характер сходимости.

Таблица 1 — исследование обусловленности метода

global_delta	ε	**значение root, возвращаемое HORDA**	**N**	**F(root)**
10^{-1}	10^{-1}	0.8000000000	2	0.0000000000
	10^{-2}	0.8000000000	2	0.0000000000
	10^{-3}	0.8000000000	2	0.0000000000
	10^{-4}	0.8000000000	2	0.0000000000
	10^{-5}	0.8000000000	2	0.0000000000
	10^{-6}	0.8000000000	2	0.0000000000
10^{-3}	10^{-1}	0.8190000000	2	0.0165619569
	10^{-2}	0.8190000000	2	0.0165619569
	10^{-3}	0.8230000000	4	0.0036704420
	10^{-4}	0.8240000000	7	0.0004293838
	10^{-5}	0.8240000000	7	0.0004293838
	10^{-6}	0.8240000000	7	0.0004293838
10^{-5}	10^{-1}	0.8193900000	2	0.0153100935

	10^{-2}	0.8193900000	2	0.0153100935
	10^{-3}	0.8193900000	4	0.0030552065
	10^{-4}	0.8242100000	7	-0.0002521747
	10^{-5}	0.8241400000	11	-0.0000249523
	10^{-6}	0.8241300000	13	0.0000075051
10^{-7}	10^{-1}	0.8193947000	2	0.0152950004
	10^{-2}	0.8193947000	2	0.0152950004
	10^{-3}	0.8231947000	4	0.0030399842
	10^{-4}	0.8242150000	7	-0.0002684063
	10^{-5}	0.8242150000	10	0.0000237335
	10^{-6}	0.8241330000	13	-0.0000022321

Из таблицы видно, что при грубом округлении Delta = 10^{-1} метод уже после двух итераций фиксируется на значении 0.8, хотя невязка исходного уравнения остается большой: f(root) ≈ 0.0763. Это означает, что слишком грубое округление полностью разрушает полезную информацию о ходе итерационного процесса.

При Delta = 10^{-3} результат уточняется только до root ≈ 0.8240, после чего дальнейшее уменьшение Eps уже не влияет на найденный корень. Следовательно, точность результата начинает определяться не требуемой точностью метода, а точностью вычисления самой функции φ(x).

При Delta = 10^{-5} и Delta = 10^{-7} удается получить приближение, близкое к точному корню. Однако и в этом случае после некоторого момента уменьшение Eps перестает давать выигрыш, так как доминирует погрешность округления значений φ(x).

Следовательно, метод простых итераций чувствителен к ошибкам в исходных данных. Для получения надежного результата необходимо выбирать Delta значительно меньше требуемой точности Eps.

По сравнению с методом Ньютона метод простых итераций проще с точки зрения вычислительной схемы, так как в самой формуле итерации не использует производную. Однако за эту простоту приходится платить меньшей скоростью сходимости и необходимостью специально подбирать удачное преобразование $x = \varphi(x)$.

Выводы.

В ходе лабораторной работы с использованием функций ITER и Round из файла methods.cpp был найден корень уравнения $\arccos(x^2) - x = 0$ методом простых итераций. Для данной функции выбран рабочий отрезок [0.7, 0.9] и эквивалентное преобразование $x = \sqrt{(\cos x)}$, для которого выполняется условие сходимости $|\varphi'(x)| \leq q < 1$, где $q \approx 0.4968$.

Исследование скорости сходимости показало, что метод сходится линейно и при уменьшении Eps от 10^{-1} до 10^{-6} число итераций увеличивается от 2 до 13. Поскольку $\varphi'(x) < 0$, последовательность приближений колеблется около корня, что подтверждается как теоретически, так и экспериментально.

Исследование обусловленности показало, что точность результата существенно зависит от точности вычисления функции $\varphi(x)$. При грубом округлении Delta дальнейшее уменьшение Eps не улучшает ответ, поэтому для практических вычислений необходимо выбирать Delta заметно меньше требуемой точности корня.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lb6.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>

#include "methods.h"

using namespace std;

double global_delta = 0.0;

double Equation(double x)
{
    return acos(x * x) - x;
}

double F(double x)
{
    double y = sqrt(cos(x));
    if (global_delta > 0.0) return Round(y, global_delta);
    return y;
}

double F1(double x)
{
    double y = -sin(x) / (2.0 * sqrt(cos(x)));
    if (global_delta > 0.0) return Round(y, global_delta);
    return y;
}

int main()
{
    double x0 = 0.8;
    int N;

    cout << fixed << setprecision(10);

    cout << "Зависимость N от Eps" << endl;
    global_delta = 0.0;
    for (double eps = 0.1; eps >= 1e-6; eps /= 10.0)
    {
        double root = ITER(x0, eps, N);
        cout << "Eps: " << eps
             << "\tN: " << N
             << "\tRoot: " << root
             << "\tf(root): " << Equation(root) << endl;
    }

    cout << endl << "Влияние Delta" << endl;
    double deltas[] = {1e-1, 1e-3, 1e-5, 1e-7};

    for (double delta : deltas)
    {
        global_delta = delta;
        cout << "Delta: " << global_delta << endl;
```

```
        for (double eps = 0.1; eps >= 1e-6; eps /= 10.0)
        {
            double root = ITER(x0, eps, N);
            cout << "Eps: " << eps
                 << "\tN: " << N
                 << "\tRoot: " << root
                 << "\tf(root): " << Equation(root) << endl;
        }

        cout << endl;
    }

    return 0;
}
```

Файл methods.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

extern double F(double);

double BISECT(double Left,double Right,double Eps,int &N)
{
  double E = fabs(Eps)*2.0;
  double FLeft = F(Left);
  double FRight = F(Right);
  double X = (Left+Right)/2.0;
  double Y;

  if (FLeft*FRight>0.0) {puts("Неверное задание интервала\n");exit(1);}
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit(1);}

  N=0;
  if (FLeft==0.0)  return Left;
  if (FRight==0.0) return Right;

  while ((Right-Left)>=E)
  {
    X = 0.5*(Right + Left);
    Y = F(X);
    if (Y == 0.0) return (X);
    if (Y*FLeft < 0.0)
       Right=X;
    else
     { Left=X; FLeft=Y; }
    N++;
  };

  return(X);
}

double Round (double X,double Delta)
{
 if (Delta<=1E-9) {puts("Неверное задание точности округления\n");exit(1);}
 if (X>0.0) return (Delta*(long((X/Delta)+0.5)));
    else    return (Delta*(long((X/Delta)-0.5)));
}

double ITER(double X0,double Eps,int &N)
```

```
{
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit (1);}
  double X1=F(X0);
  double X2=F(X1);
  N = 2;

  while( (X1 - X2)*(X1 - X2) > fabs((2*X1-X0-X2)*Eps) )
  {
    X0 = X1;
    X1 = X2;
    X2 = F(X1);
    N++;
  }

  return(X2);
}

#ifdef __NEWTON
double NEWTON (double X,double Eps,int &N)
{
  extern double F1 (double);
  double Y,Y1,DX;
  N=0;

  do
  {
    Y  = F(X);
    if (Y==0.0) return (X);

    Y1 = F1(X);
    if (Y1==0.0) {puts("Производная обратилась в ноль\n");exit(1);}

    DX=Y/Y1; X=X-DX; N++;
  }
  while (fabs(DX)>Eps);

  return (X);
}
#endif

double HORDA(double Left,double Right,double Eps,int &N)
{
  double FLeft = F(Left);
  double FRight = F(Right);
  double X,Y;

  if (FLeft*FRight>0.0) {puts("Неверное задание интервала\n");exit(1);}
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit(1);}

  N=0;
  if (FLeft==0.0)  return Left;
  if (FRight==0.0) return Right;

  do
  {
    X = Left-(Right-Left)*FLeft/(FRight-FLeft);
    Y = F(X);
    if (Y == 0.0) return (X);
    if (Y*FLeft < 0.0)
     { Right=X; FRight=Y; }
    else
     { Left=X; FLeft=Y; }
    N++;
  }
```

```
  while ( fabs(Y) >= Eps );

  return(X);
}
```

Файл methods.h

```
extern double F(double);
/*****************************************************************/
/*           Функция F (X) , задаваемая пользователем            */
/*****************************************************************/

#ifdef __NEWTON
extern double F1(double);
/*****************************************************************/
/*     Производная функции F (X) , задаваемая пользователем      */
/*****************************************************************/
#endif

double Round (double X,double Delta);
/*****************************************************************/
/*   Функция Round (X,Delta) , предназначена для округления      */
/*                    X с точностью Delta                        */
/*****************************************************************/

double BISECT(double Left,double Right,double Eps,int &N);
/*****************************************************************/
/*    Функция BISECT предназначена для решения уравнения F(X)=0  */
/*    методом деления отрезка пополам. Использованы обозначения: */
/*      Left - левый конец промежутка                            */
/*      Right - правый конец промежутка                          */
/*      Eps - погрешность вычисления корня уравнения;            */
/*      N - число итераций                                       */
/*****************************************************************/

double ITER(double X0,double Eps,int &N);
/*****************************************************************/
/*    Функция ITER предназначена для решения уравнения F(X)=X    */
/*      методом простой итерации. Использованы обозначения:      */
/*      X0 - начальное приближение корня                         */
/*      Eps - погрешность вычисления корня уравнения;            */
/*      N - число итераций                                       */
/*****************************************************************/

double HORDA(double Left,double Right,double Eps,int &N);
/*****************************************************************/
/*    Функция HORDA предназначена для решения уравнения F(x)=0   */
/*      методом хорд. Использованы обозначения:                  */
/*      Left - левый конец промежутка                            */
/*      Right - правый конец промежутка                          */
/*      Eps - погрешность вычисления корня уравнения;            */
/*      N - число итераций                                       */
/*****************************************************************/

double NEWTON (double X,double Eps,int &N);
/*****************************************************************/
/*  Функция NEWTON предназначена для решения уравнения F(X)=0    */
/*        методом касательных. Использованы обозначения:         */
/*      X - начальное приближение корня                          */
/*      Eps - погрешность вычисления корня уравнения;            */
/*      N - число итераций                                       */
```

```
/*****************************************************************/

#include "methods.cpp"
```